

Experiment-03

```
#include <stdio.h>
#include <stdlib.h>

int enterarray(int input[]);
void bubblesort(int input[], int output[], int n);
void selectionsort(int input[], int output[], int n);
void insertionsort(int input[], int output[], int n);
void quicksort(int input[], int output[], int l, int h);
int partition(int arr[], int l, int h);
void mergesort(int input[], int output[], int l, int h);
void merge(int arr[], int l, int m, int r);
void swap(int *a, int *b);

int main()
{
    int choice, i = 0, n = 0;
    int arr[100], bs[100], ss[100], qs[100], is[100], ms[100];

    do
    {
        printf("\n\t\t\t# SORTING OPERATIONS #");
        printf("\n-----");
        printf("\n1. Insert an array");
        printf("\n2. Bubble sort the array");
        printf("\n3. Selection sort the array");
        printf("\n4. Quick sort the array");
        printf("\n5. Insertion sort the array");
        printf("\n6. Merge sort the array");
        printf("\n7. Exit");
        printf("\n\nEnter your choice : ");
        if (scanf("%d", &choice) != 1) break;

        switch (choice)
        {
            case 1:
                n = enterarray(arr);
                break;
```

case 2:

```
if (n <= 0) { printf("Please insert an array first!\n"); break; }
bubblesort(arr, bs, n);
printf("The bubble sorted array is: ");
for (i = 0; i < n; i++) printf("%d ", bs[i]);
printf("\n");
break;
```

case 3:

```
if (n <= 0) { printf("Please insert an array first!\n"); break; }
selectionsort(arr, ss, n);
printf("The selection sorted array is: ");
for (i = 0; i < n; i++) printf("%d ", ss[i]);
printf("\n");
break;
```

case 4:

```
if (n <= 0) { printf("Please insert an array first!\n"); break; }
quicksort(arr, qs, 0, n - 1);
printf("The quick sorted array is: ");
for (i = 0; i < n; i++) printf("%d ", qs[i]);
printf("\n");
break;
```

case 5:

```
if (n <= 0) { printf("Please insert an array first!\n"); break; }
insertionsort(arr, is, n);
printf("The insertion sorted array is: ");
for (i = 0; i < n; i++) printf("%d ", is[i]);
printf("\n");
break;
```

case 6:

```
if (n <= 0) { printf("Please insert an array first!\n"); break; }
mergesort(arr, ms, 0, n - 1);
printf("The merge sorted array is: ");
for (i = 0; i < n; i++) printf("%d ", ms[i]);
printf("\n");
break;
```

case 7:

```
return 0;
```

```

        default:
            printf("\nInvalid Choice!\n");
        }
    } while (1);

    return 0;
}

```

```

int enterarray(int input[])
{
    int i, n;
    printf("Enter length of the array (max 100): ");
    scanf("%d", &n);
    if (n > 100) n = 100;
    if (n <= 0)
    {
        printf("Invalid array length!\n");
        return 0;
    }
    printf("Enter elements of the array: ");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &input[i]);
    }
    return n;
}

```

```

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

```

```

void bubblesort(int input[], int output[], int n)
{
    int i, j;
    for (i = 0; i < n; i++) output[i] = input[i];
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - 1 - i; j++)
        {
            if (output[j] > output[j + 1])
            {

```

```

        swap(&output[j], &output[j + 1]);
    }
}
}

```

```

void selectionsort(int input[], int output[], int n)
{
    int i, j, minindex;
    for (i = 0; i < n; i++) output[i] = input[i];
    for (i = 0; i < n - 1; i++)
    {
        minindex = i;
        for (j = i + 1; j < n; j++)
        {
            if (output[j] < output[minindex])
            {
                minindex = j;
            }
        }
        swap(&output[i], &output[minindex]);
    }
}

```

```

void insertionsort(int input[], int output[], int n)
{
    int i, j, key;
    for (i = 0; i < n; i++) output[i] = input[i];
    for (i = 1; i < n; i++)
    {
        key = output[i];
        j = i - 1;
        while (j >= 0 && output[j] > key)
        {
            output[j + 1] = output[j];
            j--;
        }
        output[j + 1] = key;
    }
}

```

```

int partition(int arr[], int l, int h)
{
    int pivot = arr[h];

```

```

int i = l - 1, j;
for (j = l; j < h; j++)
{
    if (arr[j] < pivot)
    {
        i++;
        swap(&arr[i], &arr[j]);
    }
}
swap(&arr[i + 1], &arr[h]);
return (i + 1);
}

```

```

void quicksort(int input[], int output[], int l, int h)
{
    int i;
    if (l == 0 && input != output)
    {
        for (i = 0; i <= h; i++) output[i] = input[i];
    }
    if (l < h)
    {
        int pi = partition(output, l, h);
        quicksort(output, output, l, pi - 1);
        quicksort(output, output, pi + 1, h);
    }
}

```

```

void merge(int arr[], int l, int m, int r)
{
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[100], R[100];

    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    i = 0;
    j = 0;
    k = l;
    while (i < n1 && j < n2)
    {

```

```

    if (L[i] <= R[j])
    {
        arr[k] = L[i];
        i++;
    }
    else
    {
        arr[k] = R[j];
        j++;
    }
    k++;
}

while (i < n1)
{
    arr[k] = L[i];
    i++;
    k++;
}

while (j < n2)
{
    arr[k] = R[j];
    j++;
    k++;
}
}

void mergesort(int input[], int output[], int l, int h)
{
    int i;
    if (l == 0 && input != output)
    {
        for (i = 0; i <= h; i++) output[i] = input[i];
    }
    if (l < h)
    {
        int m = l + (h - l) / 2;
        mergesort(output, output, l, m);
        mergesort(output, output, m + 1, h);
        merge(output, l, m, h);
    }
}

```


SORTING OPERATIONS

-
1. Insert an array
 2. Bubble sort the array
 3. Selection sort the array
 4. Quick sort the array
 5. Insertion sort the array
 6. Merge sort the array
 7. Exit

Enter your choice : 1

Enter length of the array (max 100): 5

Enter elements of the array: 1

2

3

4

5

SORTING OPERATIONS

-
1. Insert an array
 2. Bubble sort the array
 3. Selection sort the array
 4. Quick sort the array
 5. Insertion sort the array
 6. Merge sort the array
 7. Exit

Enter your choice : 2

Enter your choice : 2

The bubble sorted array is: 1 2 3 4 5

SORTING OPERATIONS

1. Insert an array
2. Bubble sort the array
3. Selection sort the array
4. Quick sort the array
5. Insertion sort the array
6. Merge sort the array
7. Exit

Enter your choice : 3

The selection sorted array is: 1 2 3 4 5

SORTING OPERATIONS

1. Insert an array
2. Bubble sort the array
3. Selection sort the array
4. Quick sort the array
5. Insertion sort the array
6. Merge sort the array
7. Exit

Enter your choice : 4

The quick sorted array is: 1 2 3 4 5

SORTING OPERATIONS

-
1. Insert an array
 2. Bubble sort the array
 3. Selection sort the array
 4. Quick sort the array
 5. Insertion sort the array
 6. Merge sort the array
 7. Exit

Enter your choice : 5

The insertion sorted array is: 1 2 3 4 5

SORTING OPERATIONS

1. Insert an array
2. Bubble sort the array
3. Selection sort the array
4. Quick sort the array
5. Insertion sort the array
6. Merge sort the array
7. Exit

Enter your choice : 6

The merge sorted array is: 1 2 3 4 5

SORTING OPERATIONS

1. Insert an array
2. Bubble sort the array
3. Selection sort the array

SORTING OPERATIONS

1. Insert an array
2. Bubble sort the array
3. Selection sort the array
4. Quick sort the array
5. Insertion sort the array
6. Merge sort the array
7. Exit

Enter your choice : 7

...Program finished with exit code 0

Press ENTER to exit console.